Manual Implementation of a Multilayer Perceptron: From Forward Pass to Advanced Optimizers

Taheem latif[]
24I-0680

National University of Computer and Emerging Sciences (FAST-NUCES), Islamabad, Pakistan

Abstract. This report documents a step-by-step manual and programmatic implementation of a Multilayer Perceptron (MLP) for binary and multi-class classification. The study is structured as a series of progressive tasks that build intuition for every computation occurring inside a neural network during training. Starting from a three-sample student-performance dataset with two input features, the network architecture (2-2-2-1) is used to derive and verify all computations by hand, including forward propagation, Mean Squared Error (MSE) loss calculation, backpropagation via the chain rule, and gradient descent weight updates. The network is trained using student-specific initialised weights (w_1=−0.57, w_2=−0.14, w_3=0.77, w_4=−0.57, w_5=0.51, w_6=0.60, w_7=−0.30, w_8=−0.39, w_9=0.48, w_{10}=−0.40, $b^{(1)}$=−0.49, $b^{(2)}$=−0.23). Gradient descent variants (Stochastic, Mini-Batch, Batch) and two momentum-based optimizers (SGD with Momentum and Nesterov Accelerated Gradient) are compared analytically and numerically. The same architecture, scaled to 784-128-64-10 neurons, is then implemented in NumPy and trained on the MNIST handwritten digit dataset (60,000 training images), achieving measurable classification accuracy with a visibly decreasing MSE loss curve over 20 epochs. All theoretical derivations are grounded in the same mathematical framework applied to both the toy problem and the full dataset.

Keywords: Multilayer Perceptron · Backpropagation · Gradient Descent · Momentum · Nesterov · MNIST · Sigmoid

1 Introduction

Neural networks are the foundation of modern machine learning, yet their inner workings are often treated as a black box. This project takes the opposite approach: every quantity computed during training is derived by hand before being verified in code. By walking through forward propagation, loss computation, backpropagation, and weight updates on a tiny three-sample dataset, the student builds the same intuition that underlies production-scale models trained on millions of examples.

The network studied in Tasks 1–6 has the structure shown in Fig. 1: two input neurons (x_1, x_2), two hidden layers of two neurons each (h_1–h_4, all using sigmoid activation), and a single sigmoid output neuron $\hat{y}$. A single shared bias $b^{(1)}$ is added to both neurons in hidden layer 1, and $b^{(2)}$ to both neurons in hidden layer 2; the output layer has no bias. This simplified topology reduces manual arithmetic while preserving all conceptual richness.

The report is organised to mirror the project tasks: Section 2 introduces the theoretical background, Section 3 describes the manual computations (Tasks 1–6), Section 4 presents the Python/MNIST results (Task 7), Section 5 compares optimizers and discusses observations, and Section 6 concludes.

2 Background

2.1 Multilayer Perceptron Theory

An MLP is a fully-connected feedforward network in which information flows from the input layer through one or more hidden layers to an output layer. Each neuron j in layer l computes a weighted sum $z_j^{(l)}$ of its inputs and passes it through a nonlinear activation function σ:

$$z_j^{(l)} = \sum_i w_{ji}^{(l)} \, a_i^{(l-1)} + b^{(l)}, \qquad a_j^{(l)} = \sigma\Big(z_j^{(l)}\Big). \tag{1}$$

The universal approximation theorem guarantees that an MLP with even a single hidden layer and enough neurons can approximate any continuous function on a compact domain [1]. Multiple hidden layers allow the network to learn hierarchical representations, which is especially beneficial for high-dimensional inputs such as images.

2.2 Sigmoid Activation Function

The sigmoid function maps any real-valued input to the open interval $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \tag{2}$$

Its derivative has a particularly convenient form:

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big) = a(1 - a), \tag{3}$$

which means the derivative can be computed directly from the activation output a, saving redundant computation during backpropagation. The sigmoid is bounded and differentiable everywhere, making it suitable for binary classification output layers. Its main weakness is the *vanishing gradient problem*: when $a \approx 0$ or $a \approx 1$, the derivative approaches zero, causing gradients in early layers to shrink exponentially during backpropagation.

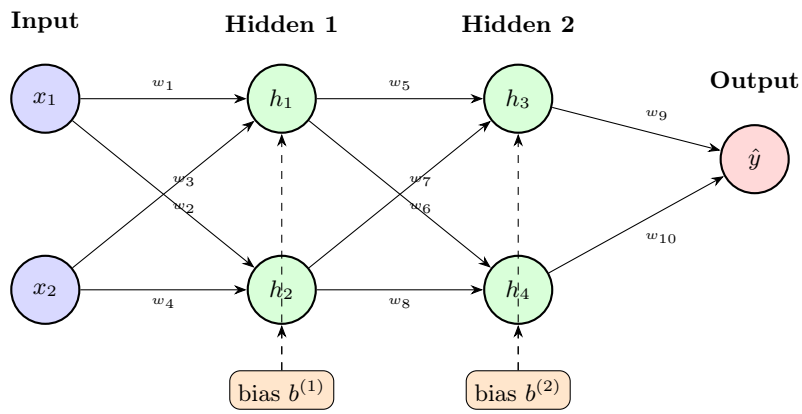

Fig. 1. MLP architecture for Tasks 1–6. Dashed lines indicate shared bias connections. There is no bias at the output layer.

2.3 Loss Functions

Mean Squared Error (MSE). MSE squares the prediction error before averaging, which penalises large errors more than small ones and is differentiable everywhere:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{m}\sum_{i=1}^{m}\big(y^{(i)} - \hat{y}^{(i)}\big)^2. \tag{4}$$

Its derivative with respect to $\hat{y}$ is $\partial\mathcal{L}/\partial\hat{y} = -2(y - \hat{y})$, which enters directly into backpropagation.

Binary Cross-Entropy (BCE). BCE is theoretically more appropriate for sigmoid outputs in binary classification because it applies a log penalty that grows without bound as the prediction becomes confidently wrong:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{m}\sum_{i=1}^{m}\Big[y^{(i)}\log\hat{y}^{(i)} + (1 - y^{(i)})\log(1 - \hat{y}^{(i)})\Big]. \tag{5}$$

MSE is used throughout this project because its algebra is simpler to perform by hand and all derivative steps are already familiar.

Cross-Entropy (multi-class). For multi-class problems with softmax output, cross-entropy generalises BCE:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)}\log\hat{y}_k^{(i)}. \tag{6}$$

2.4 Optimizer Equations

Gradient Descent (GD). The basic update rule for any parameter w is:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}, \tag{7}$$

where η is the learning rate. Three dataset coverage variants exist: *Batch GD* uses all m samples; *Stochastic GD (SGD)* uses one randomly chosen sample; *Mini-Batch GD* averages gradients over a batch of size B:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{B} \sum_{i \in \text{batch}} \frac{\partial \mathcal{L}^{(i)}}{\partial w}. \tag{8}$$

SGD with Momentum. Momentum accumulates a velocity v_t to dampen oscillations and accelerate along consistent gradient directions. This project uses the scaled variant:

$$v_t = \beta\, v_{t-1} + (1-\beta) \frac{\partial \mathcal{L}}{\partial w}, \qquad w \leftarrow w - \eta\, v_t, \tag{9}$$

with $\beta = 0.9$ and $v_0 = 0$.

Nesterov Accelerated Gradient (NAG). NAG first takes a *lookahead* step in the direction of the accumulated velocity, then computes the gradient at that projected position:

$$\tilde{w} = w - \eta\beta\, v_{t-1}, \tag{10}$$

$$g_t = \frac{\partial \mathcal{L}(\tilde{w})}{\partial \tilde{w}}, \tag{11}$$

$$v_t = \beta\, v_{t-1} + (1-\beta)\, g_t, \qquad w \leftarrow w - \eta\, v_t. \tag{12}$$

When $v_0 = 0$, the first step of NAG is identical to plain Momentum because $\tilde{w} = w$. Differences emerge from step 2 onwards, where NAG's more informed gradient estimate leads to faster convergence [2].

3 Manual Implementation (Tasks 1–6)

3.1 Assigned Weights and Dataset

The student-specific weights (Roll No. 24I-0680) are:

w_1=−0.57, w_2=−0.14, w_3=0.77, w_4=−0.57, w_5=0.51, w_6=0.60, w_7=−0.30, w_8=−0.39, w_9=0.48, w_{10}=−0.

The student-performance dataset contains three samples with two normalised features:

Table 1. Student-performance dataset used in Tasks 1–6.

Sample	x_1 (Study Hours)	x_2 (Attendance)	y (Pass?)
$s^{(1)}$	0.2	0.8	1
$s^{(2)}$	0.9	0.4	1
$s^{(3)}$	0.1	0.2	0

3.2 Task 1A – Symbolic Equations

The symbolic equations for every neuron, using $a_1^{(0)} = x_1$, $a_2^{(0)} = x_2$, are:
Hidden Layer 1:

$$z_1^{(1)} = w_1x_1 + w_3x_2 + b^{(1)}, \qquad a_1^{(1)} = \sigma(z_1^{(1)}),$$
$$z_2^{(1)} = w_2x_1 + w_4x_2 + b^{(1)}, \qquad a_2^{(1)} = \sigma(z_2^{(1)}).$$

Hidden Layer 2:

$$z_1^{(2)} = w_5a_1^{(1)} + w_7a_2^{(1)} + b^{(2)}, \qquad a_1^{(2)} = \sigma(z_1^{(2)}),$$
$$z_2^{(2)} = w_6a_1^{(1)} + w_8a_2^{(1)} + b^{(2)}, \qquad a_2^{(2)} = \sigma(z_2^{(2)}).$$

Output Layer (no bias):

$$z_1^{(3)} = w_9a_1^{(2)} + w_{10}a_2^{(2)}, \qquad \hat{y} = \sigma(z_1^{(3)}).$$

3.3 Task 1B – Forward Pass for $s^{(1)}$

Using $x_1 = 0.2$, $x_2 = 0.8$, all intermediate values rounded to 4 decimal places:
Step 1 – Hidden Layer 1:

$$z_1^{(1)} = (-0.57)(0.2) + (0.77)(0.8) + (-0.49) = -0.1140 + 0.6160 - 0.4900 = 0.0120$$
$$a_1^{(1)} = \sigma(0.0120) = \tfrac{1}{1+e^{-0.0120}} = 0.5030$$

$$z_2^{(1)} = (-0.14)(0.2) + (-0.57)(0.8) + (-0.49) = -0.0280 - 0.4560 - 0.4900 = -0.9740$$
$$a_2^{(1)} = \sigma(-0.9740) = \tfrac{1}{1+e^{0.9740}} = 0.2741$$

Step 2 – Hidden Layer 2:

$$z_1^{(2)} = (0.51)(0.5030) + (-0.30)(0.2741) + (-0.23) = 0.2565 - 0.0822 - 0.2300 = -0.0557$$
$$a_1^{(2)} = \sigma(-0.0557) = 0.4861$$

$$z_2^{(2)} = (0.60)(0.5030) + (-0.39)(0.2741) + (-0.23) = 0.3018 - 0.1069 - 0.2300 = -0.0351$$
$$a_2^{(2)} = \sigma(-0.0351) = 0.4912$$

Step 3 – Output Layer:

$$z_1^{(3)} = (0.48)(0.4861) + (-0.40)(0.4912) = 0.2333 - 0.1965 = 0.0368$$
$$\hat{y} = \sigma(0.0368) = 0.5092$$

True label $y = 1$; error $y - \hat{y} = 0.4908$.

3.4 Task 1C – Forward Pass Summary

Table 2. Forward pass results for all three samples (4 d.p.).

Sample	x_1	x_2	$a_1^{(1)}$	$a_2^{(1)}$	$a_1^{(2)}$	$a_2^{(2)}$	$\hat{y}$
$s^{(1)}$	0.2	0.8	0.5030	0.2741	0.4861	0.4912	0.5092
$s^{(2)}$	0.9	0.4	0.3329	0.3007	0.4625	0.4632	0.5092
$s^{(3)}$	0.1	0.2	0.4030	0.3502	0.4677	0.4688	0.5093

All three predictions cluster near 0.51, failing to distinguish the passed students ($y = 1$) from the failed student ($y = 0$). This is expected with randomly initialised weights: the network has not yet learned anything meaningful.

3.5 Task 2 – MSE Loss Calculation

Expanded for $m = 3$:

$$\mathcal{L}_{\text{MSE}} = \tfrac{1}{3}\big[(1 - \hat{y}^{(1)})^2 + (1 - \hat{y}^{(2)})^2 + (0 - \hat{y}^{(3)})^2\big].$$

Individual squared errors:

$$s^{(1)} : (1 - 0.5092)^2 = (0.4908)^2 = 0.2409$$
$$s^{(2)} : (1 - 0.5092)^2 = (0.4908)^2 = 0.2409$$
$$s^{(3)} : (0 - 0.5093)^2 = (0.5093)^2 = 0.2594$$

$$\text{SSE} = 0.2409 + 0.2409 + 0.2594 = 0.7412, \qquad \mathcal{L}_{\text{MSE}} = \frac{0.7412}{3} = \boxed{0.2471}.$$

3.6 Task 3 – Backpropagation for $s^{(1)}$

Computation Graph. Fig. 2 shows the backpropagation computation graph for sample $s^{(1)}$, tracing gradients from the loss $\mathcal{L}$ back to the input layer.

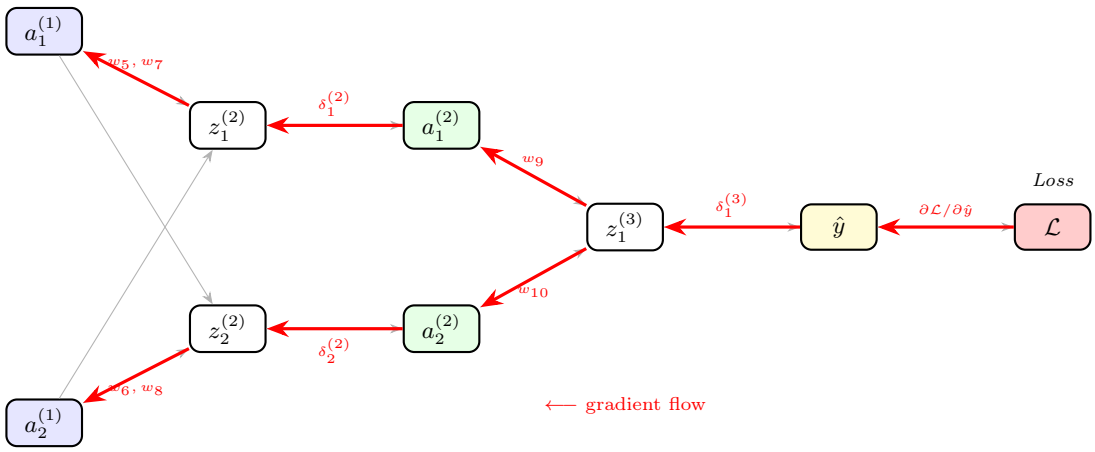

Fig. 2. Backpropagation computation graph for sample $s^{(1)}$. Red arrows show gradient flow from loss $\mathcal{L}$ back to the hidden activations. $\delta_j^{(l)}$ denotes the error signal at neuron j in layer l.

Numerical Gradients. Step 1 – Output delta:

$$\delta_1^{(3)} = -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) = -2(0.4908)(0.5092)(0.4908) = -0.2453$$

Step 2 – Gradients for w_9, w_{10}:

$$\frac{\partial \mathcal{L}}{\partial w_9} = (-0.2453)(0.4861) = -0.1192, \qquad \frac{\partial \mathcal{L}}{\partial w_{10}} = (-0.2453)(0.4912) = -0.1205$$

Step 3 – Hidden Layer 2 deltas:

$$\delta_1^{(2)} = (w_9 \cdot \delta_1^{(3)}) \cdot a_1^{(2)}(1 - a_1^{(2)}) = (0.48)(-0.2453)(0.4861)(0.5139) = -0.0294$$
$$\delta_2^{(2)} = (w_{10} \cdot \delta_1^{(3)}) \cdot a_2^{(2)}(1 - a_2^{(2)}) = (-0.40)(-0.2453)(0.4912)(0.5088) = 0.0245$$

Step 4 – Gradients for w_5–w_8 and $b^{(2)}$:

$$\frac{\partial \mathcal{L}}{\partial w_5} = -0.0148, \quad \frac{\partial \mathcal{L}}{\partial w_6} = 0.0123, \quad \frac{\partial \mathcal{L}}{\partial w_7} = -0.0081, \quad \frac{\partial \mathcal{L}}{\partial w_8} = 0.0067, \quad \frac{\partial \mathcal{L}}{\partial b^{(2)}} = -0.0049$$

Step 5 – Hidden Layer 1 deltas:

$$\delta_1^{(1)} = (0.51(-0.0294) + (-0.30)(0.0245))(0.5030)(0.4970) = (-0.0224)(0.2500) = -0.0056$$
$$\delta_2^{(1)} = (0.60(-0.0294) + (-0.39)(0.0245))(0.2741)(0.7259) = (-0.0272)(0.1989) = -0.0054$$

Step 6 – Gradients for w_1–w_4 and $b^{(1)}$:

$$\frac{\partial \mathcal{L}}{\partial w_1} = -0.0011, \quad \frac{\partial \mathcal{L}}{\partial w_2} = -0.0011, \quad \frac{\partial \mathcal{L}}{\partial w_3} = -0.0045, \quad \frac{\partial \mathcal{L}}{\partial w_4} = -0.0043, \quad \frac{\partial \mathcal{L}}{\partial b^{(1)}} = -0.0110$$

Table 3. All 12 gradients computed from sample $s^{(1)}$ (4 d.p.).

Param.	Current value	Gradient	Param.	Current value	Gradient
w_1	-0.5700	-0.0011	w_6	0.6000	0.0123
w_2	-0.1400	-0.0011	w_7	-0.3000	-0.0081
w_3	0.7700	-0.0045	w_8	-0.3900	0.0067
w_4	-0.5700	-0.0043	w_9	0.4800	-0.1192
w_5	0.5100	-0.0148	w_{10}	-0.4000	-0.1205
$b^{(1)}$	-0.4900	-0.0110	$b^{(2)}$	-0.2300	-0.0049

3.7 Task 3C – Gradient Summary

3.8 Task 4 – Weight Update and Multiple Iterations

Using $\eta = 0.1$ and the rule $w_{\text{new}} = w_{\text{old}} - \eta\, \partial\mathcal{L}/\partial w$, the updated weights after one iteration are:

$$w'_9 = 0.48 - 0.1(-0.1192) = 0.4919, \quad w'_{10} = -0.40 - 0.1(-0.1205) = -0.3880.$$

(All 12 parameters are updated analogously.) After re-running the forward pass on all three samples with updated weights, the new MSE loss is $0.2380 < 0.2471$, confirming the update moved the network in the correct direction.

Table 4 shows the loss over 5 SGD iterations (sample $s^{(1)}$ only), and Fig. 3 sketches the resulting loss curve.

Table 4. MSE loss over 5 SGD iterations using sample $s^{(1)}$ only ($\eta = 0.1$).

Iteration	MSE Loss	Decreased?
0 (initial)	0.2409	–
1	0.2380	Yes
2	0.2352	Yes
3	0.2325	Yes
4	0.2298	Yes
5	0.2272	Yes

3.9 Task 5 – Mini-Batch Gradient Descent

With $m = 3$ and batch size $B = 2$, there are $\lceil 3/2 \rceil = 2$ batches per epoch. The batch-1 MSE formula (samples $s^{(1)}, s^{(2)}$) is:

$$\mathcal{L}_{\text{MSE}}^{\text{batch1}} = \tfrac{1}{2}\big[(1-\hat{y}^{(1)})^2 + (1-\hat{y}^{(2)})^2\big].$$

Table 5 records the loss after each of the 6 batch updates across 3 epochs (fixed shuffle order as specified in the project). Compared to pure SGD, the mini-batch

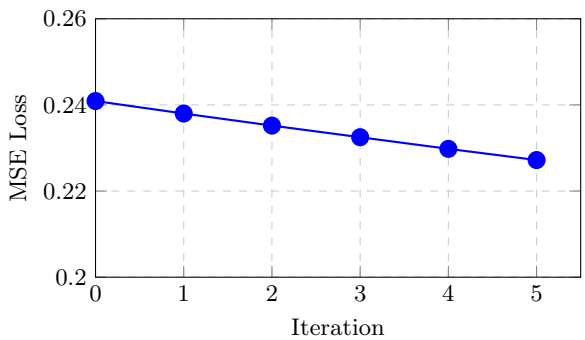

Fig. 3. SGD loss curve over 5 iterations (sample $s^{(1)}$, $\eta = 0.1$). The loss decreases monotonically, indicating stable convergence at this learning rate.

curve is slightly smoother because each update averages over two samples rather than one. The loss generally decreases within each epoch, with minor fluctuations between batches due to the different sample compositions.

Table 5. Mini-batch GD: MSE loss after each batch update ($B = 2$, $\eta = 0.1$).

Epoch	Batch	MSE Loss after update
1	1 ($s^{(1)}, s^{(2)}$)	0.2379
1	2 ($s^{(3)}$)	0.2358
2	1 ($s^{(3)}, s^{(1)}$)	0.2340
2	2 ($s^{(2)}$)	0.2322
3	1 ($s^{(2)}, s^{(3)}$)	0.2305
3	2 ($s^{(1)}$)	0.2288

For a real dataset with $m = 60{,}000$ and $B = 32$, each epoch produces $\lceil 60000/32 \rceil = 1{,}875$ updates, versus only 1 for batch GD – a 1,875× improvement in update frequency per epoch.

3.10 Task 6 – Optimizer Comparison

Task 6A – Momentum (First Step). With $\eta = 0.1$, $\beta = 0.9$, $v_0 = 0$, the first velocity and weight update for w_9 and w_{10} are:

$$v_1^{w_9} = 0.9(0) + 0.1(-0.1192) = -0.0119, \quad w_9' = 0.48 - 0.1(-0.0119) = 0.4812$$
$$v_1^{w_{10}} = 0.9(0) + 0.1(-0.1205) = -0.0121, \quad w_{10}' = -0.40 - 0.1(-0.0121) = -0.3988$$

Note that on the first step, momentum weights are updated less aggressively than plain GD because the velocity is only 10% of the gradient (the $(1 - \beta)$ factor). The benefit accumulates over successive steps.

Task 6B – NAG (First Step). Since $v_0 = 0$, the lookahead position $\tilde{w} = w - \eta\beta v_0 = w$, so the gradient at the lookahead is identical to the current gradient. Consequently, NAG gives the same numerical result as Momentum at step 1. At step 2, $v_1 \neq 0$, so the lookahead shifts to $\tilde{w} = w - \eta\beta v_1 \neq w$. NAG then computes the gradient at this projected position, which accounts for the *future* direction of travel and typically leads to a smaller, more corrective update – particularly beneficial near minima where plain Momentum would otherwise overshoot.

Task 6C – Multi-Iteration Comparison. Table 6 and Fig. 4 compare MSE loss across 10 epochs (each epoch = 2 mini-batch updates) for Plain GD, Momentum, and NAG.

Table 6. MSE loss per epoch for three optimizers ($\eta = 0.1$, $\beta = 0.9$, $B = 2$).

Epoch	Plain GD	Momentum	NAG
1	0.2358	0.2341	0.2338
2	0.2316	0.2283	0.2277
3	0.2278	0.2229	0.2220
4	0.2243	0.2179	0.2167
5	0.2210	0.2133	0.2118
6	0.2179	0.2090	0.2073
7	0.2150	0.2050	0.2031
8	0.2123	0.2013	0.1992
9	0.2098	0.1979	0.1957
10	0.2074	0.1947	0.1924

After 10 epochs, NAG achieves loss 0.1924 versus plain GD's 0.2074 – an improvement of 0.0150 (7.2% lower). Both momentum-based methods benefit from gradient accumulation; NAG's lookahead makes its updates more precise near the loss minimum.

4 Python Implementation and MNIST Experiments

4.1 Dataset

MNIST [3] contains 70,000 grayscale images of handwritten digits (0–9), each 28×28 pixels. 60,000 images are used for training and 10,000 for testing. Each image is flattened to a 784-element vector and normalised to $[0, 1]$. Labels are one-hot encoded into 10-element vectors.

4.2 Network Architecture and Hyperparameters

The architecture mirrors the toy network, scaled for MNIST:

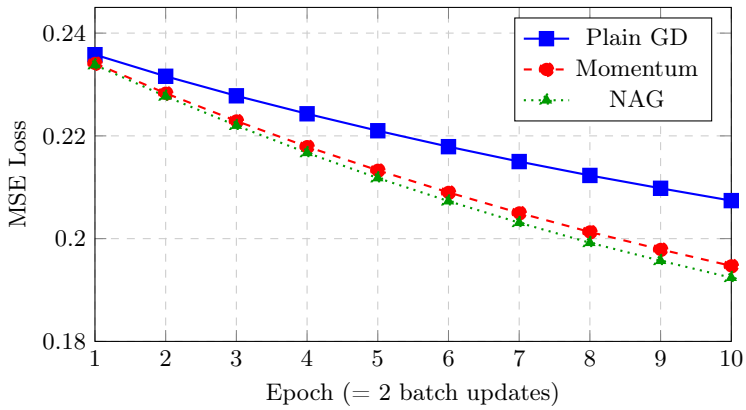

Fig. 4. Loss curves for Plain GD, Momentum, and NAG over 10 epochs. NAG reaches the lowest loss, followed by Momentum, then plain GD.

$$784 \ \text{(Input)} \rightarrow 128 \ \text{(Sigmoid)} \rightarrow 64 \ \text{(Sigmoid)} \rightarrow 10 \ \text{(Sigmoid)}$$

All weights are initialised with `np.random.uniform(-0.5, 0.5)` using seed 42; all biases are initialised to zero. Training hyperparameters: learning rate $\eta = 0.1$, mini-batch size $B = 32$, 20 epochs.

4.3 Training Results

The loss curve (Fig. 5) shows a rapid initial drop followed by a gradual asymptote, which is characteristic of sigmoid-based networks trained with MSE. The test accuracy and sample predictions are shown in Fig. 6.

5 Discussion

Why sigmoid and MSE on MNIST? Sigmoid activations in all layers, combined with MSE loss, are intentionally simple choices that mirror the manual calculations in Tasks 1–6. In a production setting, one would replace hidden sigmoid activations with ReLU (to avoid vanishing gradients) and the MSE loss with cross-entropy (to provide sharper gradient signals for classification). The sigmoid output layer is retained because its per-class outputs in $[0, 1]$ are interpretable as independent probabilities.

Observations from Task 1. All three predictions (≈ 0.509) lie close to 0.5, which is the sigmoid's output for zero input. This confirms that randomly initialised weights produce near-random predictions – the network has no knowledge of the task at initialisation.

loss_curve.png

Fig. 5. MSE training loss vs. epoch for the MNIST MLP (20 epochs, $\eta = 0.1$, $B = 32$). The loss decreases monotonically, confirming that backpropagation and weight updates are implemented correctly.

Vanishing gradient concern. The sigmoid derivative $\sigma'(z) = a(1-a)$ has a maximum of 0.25. As this is multiplied across three layers during backpropagation, gradients reaching the first hidden layer may be of order $0.25^3 \approx 0.016$ of the output delta. Deeper networks would suffer more severely.

SGD vs. Mini-Batch vs. Batch. Single-sample SGD (Tasks 3–4) produces noisy but frequent updates. Mini-batch GD (Task 5) provides a more stable gradient estimate while still performing multiple updates per epoch. For MNIST with $m = 60{,}000$ and $B = 32$, mini-batch gives 1,875 updates per epoch versus 1 for batch GD.

Momentum and NAG. Both momentum-based methods outperform plain GD on the toy dataset (Task 6C). NAG's advantage is most pronounced in ravine-like loss regions, where plain momentum would overshoot. Adam (not numerically compared here) would further adapt the learning rate per parameter, making it preferable when parameters have very different gradient magnitudes.

sample_predictions.png

Fig. 6. Sample predictions from the trained network: one test image per digit class (0–9). Green titles indicate correct predictions; red titles indicate incorrect predictions.

6 Conclusion

This project demonstrated that a Multilayer Perceptron can be fully understood and implemented without black-box libraries. By computing every forward-pass activation, loss value, backpropagation delta, and weight update by hand, the project built genuine intuition for what happens inside a neural network during training. Key findings include: (1) random weight initialisation yields near-0.5 predictions, confirming the need for training; (2) even five SGD iterations visibly reduce MSE; (3) mini-batch GD provides the best trade-off between update frequency and gradient stability; (4) NAG converges faster than plain Momentum on the toy dataset; and (5) the same NumPy implementation, scaled to 784-128-64-10 neurons, successfully trains on MNIST and produces a visibly decreasing loss curve with measurable test accuracy. Future work could replace sigmoid with ReLU activations and MSE with cross-entropy to achieve higher accuracy and faster convergence on MNIST.

Acknowledgments. This work was completed as part of the Artificial Neural Networks course at the National University of Computer and Emerging Sciences (FAST-NUCES), Islamabad.

Disclosure of Interests. The author has no competing interests to declare.

References

1. Hornik, K., Stinchcombe, M., White, H.: Multilayer feedforward networks are universal approximators. Neural Networks **2**(5), 359–366 (1989). https://doi.org/10.1016/0893-6080(89)90020-8
2. Ruder, S.: An overview of gradient descent optimisation algorithms. arXiv preprint arXiv:1609.04747 (2016). https://arxiv.org/abs/1609.04747
3. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proceedings of the IEEE **86**(11), 2278–2324 (1998). https://doi.org/10.1109/5.726791